

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – CODING GOLF COMPETITION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 1 – Coding Golf Competition
Weightage:	35% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	Murshidha M	Reg. No.:	192571007
Section / Batch:	A	Date:	17/08/2024

Code of Conduct

I Murshidha M (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Murshidha

Instructions

- Log in to the assigned coding platform - Code chef (<https://www.codechef.com/skill-test/c-language-dsa>). Total: 100 Marks.
- You can only attempt this assessment once. Also, you will not be able to pause the assessment after starting.
- You are not allowed to switch tabs during the assessment, otherwise you will be blocked from the assessment.
- You will get a detailed report on your performance at the end of the assessment.

Section / Topic	Focus	Q Nos	Marks
Data structures	Code Chef	28	100
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Accurately interprets all problem requirements and constraints before coding.	Understands most requirements with minor omissions.	Partially understands the problem; misses some constraints.	Misinterprets the problem or ignores key requirements.
Correctness of Solution	30	Program compiles successfully and passes all hidden and visible test cases.	Program passes most test cases with minor errors.	Program passes some test cases but has logical issues.	Program fails to compile or produces incorrect results.
Data Structure & Algorithm Selection	20	Selects the most appropriate data structure and algorithm with an optimized solution.	Chooses an appropriate solution with minor inefficiencies.	Uses a workable but less efficient approach.	Uses an inappropriate data structure or algorithm.
Code Quality & Documentation	20	Well-structured, modular, readable, and properly indented code with meaningful identifiers.	Readable code with minor formatting issues.	Basic coding style with inconsistent formatting.	Poorly organized code that is difficult to understand.
Time & Space Optimization	10	Demonstrates excellent optimization and correctly analyses time and space complexity.	Good optimization with minor inefficiencies.	Limited optimization and incomplete complexity analysis.	No optimization and incorrect or missing complexity analysis.

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Statement

What is the time complexity of this code?

```
i = 1
while i * i <= n:
    print(i)
    i += 1
```

[Explain](#)

```
i = 1
while i <= n:
    print(i)
    i *= 2
```

Select the correct answer

☐ $O(\sqrt{n})$

☒ $O(\sqrt{n} * \log(n))$

☐ $O(n)$

☐ $O(\log(n))$

Did you like the problem?

38 users found this helpful



Ctrl + J

Submit

Next

What is the time complexity of this code?

```
for i = 1 to (n / 10)
  for j = 1 to (2 * m)
    for l = 1 to k
      print((i * j) + 1)
```

See Solution

Select the correct answer

☐ $O(n+mk)$

☐ $O(nm + k)$

☐ $O(n + m + k)$

☒ $O(nmk)$

Did you like the problem?

19 users found this helpful



Ctrl + /

Submit

Statement

What is the time complexity of this code?

```
s = 0
for i = 1 to n:
    s += i

for j = 1 to s:
    print(j)
```

[Explain](#)

Select the correct answer

☒ $O(n^2)$

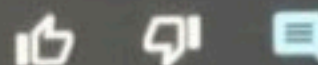
☐ $O(n * \log(n))$

☐ $O(n)$

☐ $O(n * \sqrt{n})$

Did you like the problem?

46 users found this helpful



Ctrl + ?

Submit

What is the output of the following C code?

```
#include <stdio.h>

int main(){
    int numbers[] = {1, 2, 3, 4, 5};
    printf("%d", numbers[5]);
    return 0;
}
```

Expand

Select the correct answer

☐ 0

☐ 5

☒ Garbage value

☐ Compilation error

Did you like the problem?

7 users found this helpful



Close

Submit

What is the output of the following C code?

```
#include <stdio.h>

int main() {
    int arr[6] = {1, 2, 3, 4, 5, 6};

    printf("%d", arr[2] * arr[4] % arr[1]);

    return 0;
}
```

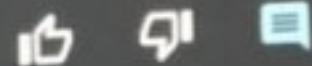
[Explain](#)

Select the correct answer

☒ 1☐ 2☐ 3☐ 4

Did you like the problem?

2 users found this helpful



0/1 / 100%

Submit

Which matrix is obtained by interchanging its rows and columns?

Which matrix is obtained by interchanging its rows and columns?

☒ A) Transpose Matrix

☐ B) Diagonal Matrix

☐ C) Scalar Matrix

☐ D) Identity Matrix

Did you like the problem?

22 users found this helpful



01 / 36

Submit

What will be the output of the given C code for `int arr[] = {64, 25, 12, 22, 11}`; and `n = 5` after 3rd pass?

```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n-1; i++) {  
        for (int j = 0; j < n-i-1; j++) {  
            if (arr[j] > arr[j+1]) {  
                // Swap arr[j] and arr[j+1]  
                int temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
            }  
        }  
    }  
}
```

[Explain](#)

Select the correct answer

☐ 25 12 22 11 64

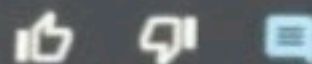
☐ 12 22 11 25 64

☒ 12 11 22 25 64

☐ 11 12 22 25 64

Did you like the problem?

2 users found this helpful



Submit

Statement

Rearrange the shuffled code to generate correct code for finding middle element of linked list.

Function Signature:

```
struct Node* findMiddleElement(struct Node* head) {  
    // code  
}
```

Sample 1:

Input	Output
List = {1, 2, 3, 4, 5, 6, 7, 8}	5

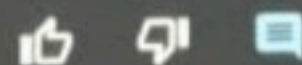
Explanation:

The findMiddleElement function uses two pointers, slow and fast, to traverse the linked list. The slow pointer moves one step at a time, while the fast pointer moves two steps at a time. When the fast pointer reaches the end of the list, the slow pointer will be at the middle element = 5.

Rearrange the following code

```
if (head == NULL || head->next == NULL) return head;  
struct Node* slow = head;  
struct Node* fast = head;  
while (fast != NULL && fast->next != NULL)  
{  
    slow = slow->next;  
    fast = fast->next->next;  
}  
return slow;
```

Did you like the problem?



Submit

Next